Round 1 prototype: around the payment

Startup and registration, enroll and refresh, issue and redeem, recovery from the seed, and what every crash leaves behind. Same notation as `payment-flow.pdf`; the attestation steps 1 to 4 of a payment are written **Attest** here.

blue sealed to the attested channel key · bank check run by the bank on its own · enclave check that rests on the enclave · P2 property · key key used · file persisted · **Attest** = `BankRequest::Attest`$\{nonce, owner\}$ → `EnclaveRequest::Attest` → `ChannelKeys::issue` → `Attestation`$\{doc, manifest\}$ → `verify_bundle` (freshness) and `check_manifest`, exactly as in the payment · every settling operation runs **B1** `claim`, **B2** owner signature, **B4a**, `Check`, **B3**, **B4b** `stage`, group commit, as in the payment

A Startup and enclave registration

❶ `tv-bank init --data D --reserve N` (`init::init`)
builds a staging dir `D.init-<pid>`: `state.key` `state key`, `issuance.key` `issuance key`, `bank-wallet.json` (a `tv_client` wallet whose index 0 state has balance N, signed with the issuance key), `bank-genesis.json` (that signed state, private), `log.jsonl` with only the header `{"log":"flights"}`; fsync, rename to `D`, fsync the parent. A non empty `D` is refused. Total supply = N.

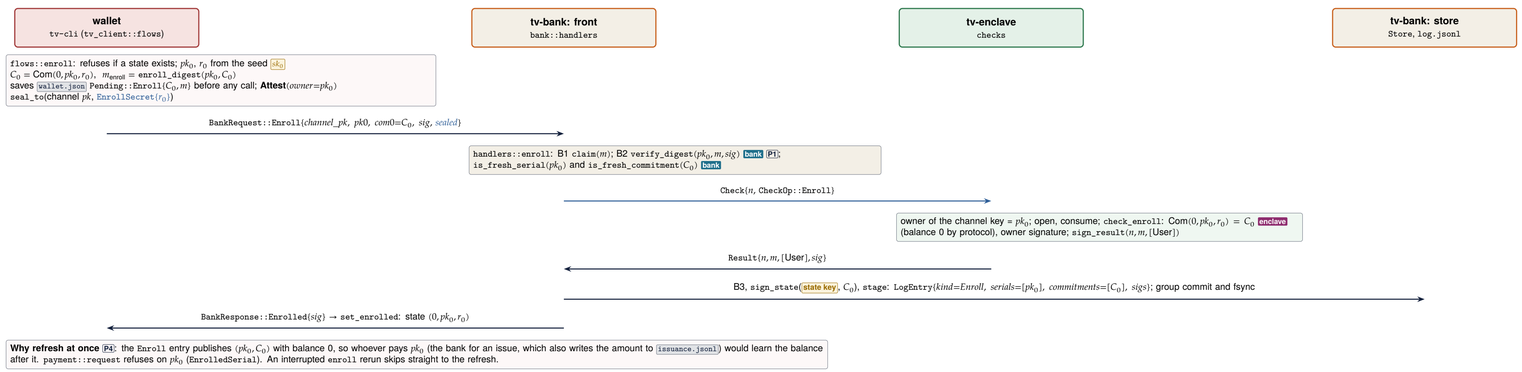
❷ `tv-enclave --mock --state-pk .. --issuance-pk ..` (vsock and NSM without `--mock`)
`Enclave::boot`: `result key` `SigningKey::generate`, memory only, new at every boot; `BankVerifiers` from both bank public keys; `user_data` = `PROTOCOL_TAG` ‖ `manifest_digest`(result pk, state pk, issuance pk); empty `ChannelKeys`. Runs on a zeroizing global allocator.

❸ `tv-bank serve --data D --enclave ADDR --allow-mock` or `--pcr0 HEX` (policy checked at startup)
`Bank::open`: `require_complete` refuses a dir missing a key, the bank wallet or the log (never recreates an empty log); `Store::open` → `flight::replay`(`log.jsonl`) rebuilds the `Index`; reads `enclave.pk` if present; listens on 7700.

❹ `tv-bank register-enclave` → `BankRequest::RegisterEnclave` (accepted from loopback peers only, carries no policy)
`Bank::register` (single flight lock) → `registry::attest_enclave`: `EnclaveRequest::Attest`$\{nonce,\ owner=$None$\}$: the enclave attests a throwaway channel key and keeps nothing → `Attestation`$\{doc, manifest, channel_pk\}$
verifier under the `serve` policy (PCR0 or mock on loopback), nonce, manifest digest, `manifest.bank` = own keys → `write_public`: `enclave.pk` (temporary file, fsync, rename, dir fsync) → `Registered`$\{result_pk\}$

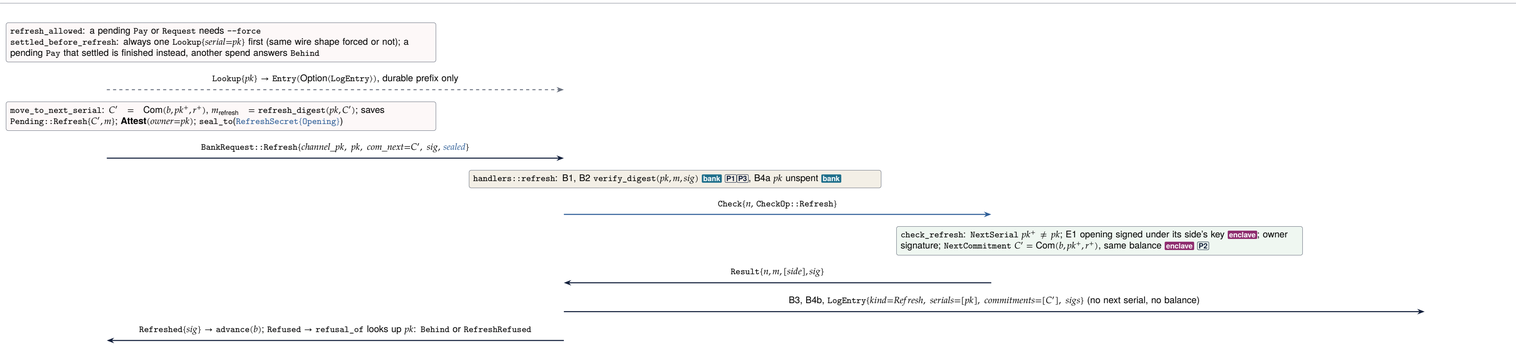
❺ **Re-registration.** When a `Result` fails B3 under the registered key, `Bank::re_register` attests the same endpoint again under the same policy, then verifies once more: a restarted enclave of the pinned image is followed without an operator; a result under any other key is refused; a transient failure answers `Unavailable`. Every bank to enclave call is bounded by `ENCLAVE_CALL_TIMEOUT` (10 s).

B `tv-cli enroll` = `flows::enroll_and_refresh`: enroll pk_0, then move off it at once



Why refresh at once P4: the `Enroll` entry publishes (pk_0, C_0) with balance 0, so whoever pays pk_0 (the bank for an issue, which also writes the amount to `issuance.jsonl`) would learn the balance after it. `payment::request` refuses on pk_0 (`EnrolledSerial`). An interrupted `enroll` rerun skips straight to the refresh.

`flows::refresh` (also `tv-cli refresh`, and `refresh --force` to burn a serial exposed by an abandoned payment)



D Issue and redeem: ordinary payments from and to the bank wallet

Redeem: the bank wallet is the payee

Total issued = the sum over `issuance.jsonl` lines whose m appears in `log.jsonl` (the ledger can over record, never under record). The ledger is not public: an issued amount is the bank's own business as payer.

E Recovery from the seed alone

❶ `tv-cli new --seed S --state-pk .. --issuance-pk ..` then `tv-cli recover` (`recover::recover`)
`BankClient::full_log`: `BankRequest::LogRange`$\{from,\ limit{=}256\}$ pages → `Entries`(`Vec<LogEntry>`) until a short page. The bank only sees whole log downloads: it never learns which serials belong to one wallet (`Lookup` is never used here) P4

❷ `walk`, locally: `by_serial` maps each spent serial to its entry; start at $pk_0 = $ `key_at(0)`
`Enroll` of pk_0 → `enrolled_state`: Com$(0, pk_0, r_0)$ must be in the entry
each `Pay` or `Refresh` spending pk_i → `step`: with pk_{i+1}, r_{i+1}, `matching_balance` tries the hint, the previous balance, then $\pm1, \pm2, \ldots$ up to $B_{\max}$ until Com(b', pk_{i+1}, r_{i+1}) is one of the entry's commitments; `sig_for` gives σ, `verify_new_state` under the bank key
stops at the first unspent serial; the wallet is restored to that index and state, any pending record is cleared

❸ `tv-bank recover` (`recover_from_genesis`): the bank wallet has no `Enroll`; it starts from `bank-genesis.json` (balance = reserve) and uses `issuance.jsonl` as `Debits` ($m \mapsto$ amount): for an issued m the hint is previous $-$ amount, otherwise the same outward search (a redeem moves at most $B_{\max}$)

Cost: one full log download, at most $2B_{\max}$ hashes per step (instant for small payments). A modified client that parked its next state on a spent serial stops recovery with an error; an `Enroll` of an already funded pk_i is skipped.

F Crashes and retries P5: nothing is lost or counted twice

Failure	What happens in the code	Recovery and test
lost response, refused connect	`retry::with_backoff`: up to 12 attempts (6 on refused), 100 ms doubling to 2 s	same request, idempotent by m
enclave dies before answering	link error → `Unavailable`; nothing staged; the restarted enclave has a new result key and an empty `ChannelKeys`	`tv-cli request` (no amount) re-attests, payer reseals; first result re-registers (B3 mismatch); `p5_crash.rs`, `crash-demo.sh`
`TV_CRASH_BEFORE_ANSWER` (exit 86)		
bank dies after the fsynced append, before the answer	restart replays `log.jsonl`; the client's resend hits **B1**: stored answer, no second entry	automatic; `crash-demo.sh`
`TV_CRASH_AFTER_APPEND` (exit 87)		
bank dies after the enclave check, before the flight is durable	staged entry lost; the resend misses B1 and the enclave refuses (key consumed); `BankClient::call` counts sends: `Refused` after more than one send of `Receive`, `Enroll`, `Refresh` reads as `Unavailable`	re-attest and reseal; `p5_crash.rs`
write or fsync of a flight fails	`Store::roll_back`: truncate to the durable length, rebuild the `Index`, tickets `RolledBack` → `Unavailable`; if that fails the store is poisoned: `Refused` until restart	re-attest and reseal; `bank/tests.rs`, `commit.rs`
power loss inside a flight	`flight::replay`: the final unconfirmed flight (zero bytes, unterminated line, or gapless complete lines without a trailer) is truncated; damage before the last valid trailer refuses to start	only never answered entries drop; `flight/tests.rs`
copies of one request at once	`InFlight::try_claim`(m): copies wait on a `watch`, then read the stored answer	`duplicate_in_flight.rs`
two spends of one serial in flight	**B4b** in `Bank::stage` re-checks the spent set under the store lock	exactly one settles; `b4b_race.rs`
receipt cannot be written	`receive` writes `receipt.json` before `commit_receive`; the wallet keeps its `Pending::Request`	rerun: B1 answers `Settled`
payer lost the receipt	`finish` without `--receipt`: Lookup(pk_A), `lookup_sig` checks the digest	or `refresh`: finishes a settled `Pay`